

5

Computing in $\mathbb{Z}/(n)$

In this chapter we introduce the finite ring $\mathbb{Z}/(n)$, the ring of integers modulo n , and the finite group of its invertible elements, $(\mathbb{Z}/(n))^*$. We discuss some of the basic algorithms for computing in these structures: the power algorithm, the Chinese remainder theorem, and algorithms for computing the Legendre and Jacobi symbols.

5.1 Basics

Recall from section 2.8.2 the following notation: if R is a ring, then R^* denotes the multiplicative group of invertible elements (units).

$\mathbb{Z}/(n)$ is a commutative ring, with the equivalence class of 1 playing the role of the identity. Its group of units is $(\mathbb{Z}/(n))^*$.

The least positive exponent e such that $a^e \equiv 1 \pmod{n}$ is called the *order* of a modulo n . We write $e = \text{ord}_n a$.

We now recall the theorem of Fermat-Euler, discussed in Chapter 2. In Exercise 14 we gave an elementary proof; for the sake of variety, we give another proof here.

THEOREM 5.1.1 Let $a \in (\mathbb{Z}/(n))^*$, and let $\varphi(n)$ be the Euler phi function. Then

$$a^{\varphi(n)} \equiv 1 \pmod{n}. \quad (5.1)$$

Proof By group theory we know that the order of any element divides the order of the group. But the order of $(\mathbb{Z}/(n))^*$ is $\varphi(n)$. ■

Unless otherwise stated, all algorithms in this section take as inputs the canonical representative $a \bmod n$ of an equivalence class, modulo n , and return canonical representatives as output. See equation (2.1).

5.2 Addition, Subtraction, Multiplication

THEOREM 5.2.1 We can add and subtract elements of $\mathbb{Z}/(n)$ in $O(\lg n)$ bit operations. We can multiply elements of $\mathbb{Z}/(n)$ using $O((\lg n)^2)$ bit operations.

Proof We only need to observe that if $0 \leq a, b < n$, then

$$(a + b) \bmod n = \begin{cases} a + b, & \text{if } a + b < n; \\ a + b - n, & \text{if } a + b \geq n. \end{cases}$$

Thus no division is necessary for modular addition. In a similar fashion, no division is necessary for modular subtraction. The bound for multiplication modulo n is clear. ■

5.3 Multiplicative Inverse

THEOREM 5.3.1 Let $d = \gcd(a, n)$. Then the congruence $ax \equiv b \pmod{n}$ has a solution x iff $d \mid b$. The solution is unique, modulo n/d .

Proof Suppose x_0 is a solution. Then $ax_0 - b = kn$ for some k . Thus $ax_0 - kn = b$. Since $d \mid a$ and $d \mid n$, we see that $d \mid b$.

Suppose $d \mid b$. Then by Theorem 4.3.1 we can find integers x_1, y_1 such that $ax_1 + ny_1 = d$. But this implies that $ax_1c + ny_1c = b$, for $c = b/d$. Now put $x_0 = x_1c$. Then $ax_0 \equiv b \pmod{n}$.

To prove the last sentence of the theorem, suppose x_0 and x_1 are both solutions to $ax \equiv b \pmod{n}$. Then $(a/d)x_0 \equiv (a/d)x_1 \pmod{n/d}$. Since $\gcd(a/d, n/d) = 1$, we can multiply by $(a/d)^{-1} \pmod{n/d}$ to obtain $x_0 \equiv x_1 \pmod{n/d}$. ■

COROLLARY 5.3.2 Let $d = \gcd(a, n)$. If $d \mid b$, we can solve $ax \equiv b \pmod{n}$ using $O((\lg n)^2)$ bit operations. In particular, if $d = 1$, by setting $b = 1$ we can find the multiplicative inverse of $a \pmod{n}$ using $O((\lg a)(\lg n))$ bit operations.

Proof This follows immediately from Theorem 4.3.2. ■

We need a notation for the canonical representative of the multiplicative inverse of $a \pmod{n}$, where $\gcd(a, n) = 1$. By abuse of notation, we will use $a^{-1} \pmod{n}$.

5.4 The Power Algorithm

In later chapters we will need to evaluate $a^e \pmod{n}$. Repeated multiplication by a gives an algorithm that uses $\Omega(e(\lg n)^2)$ bit operations, which is not satisfactory for large e .

To improve the running time, we introduce one of the most important algorithms in computational number theory: the “power algorithm” or “binary method” for exponentiation. This algorithm computes $a^e \pmod{n}$, using $O((\lg e)(\lg n)^2)$ bit operations.

It is based on the following observation:

$$a^e = \begin{cases} a \cdot a^{e-1}, & \text{if } e \text{ is odd} \\ (a^{e/2})^2, & \text{if } e \text{ is even.} \end{cases}$$

From this we can easily derive a recursive algorithm for $a^e \pmod{n}$:

POWER(a, e, n)

{ a, e, n are integers with $e \geq 0, n \geq 2$ }

if $e = 0$ then return (1)

```

else if  $e \bmod 2 = 0$  then
     $t \leftarrow \text{POWER}(a, e/2, n)$ 
    return  $(t^2 \bmod n)$ 
else
     $t \leftarrow \text{POWER}(a, e - 1, n)$ 
    return  $(at \bmod n)$ 

```

THEOREM 5.4.1 Let a, e, n be integers with $n \geq 2$, $e \geq 0$, and $0 \leq a < n$. Let $s_2(e)$ denote the number of 1's in the binary expansion of e . Then the algorithm POWER computes $a^e \bmod n$, using $(\lg e) - 1$ squarings modulo n and $s_2(e)$ multiplications by a , modulo n . Hence POWER uses $O((\lg e)(\lg n)^2)$ bit operations to compute $a^e \bmod n$.

Proof The proof is by induction on e . Clearly the result is true for $e = 0$. Now assume it is true for all $f < e$; we wish to prove it for e . If e is odd, say $e = 2k + 1$, then the algorithm first computes a^{2k} (which takes $(\lg 2k) - 1$ squarings and $s_2(2k)$ multiplications by a , by induction) and then multiplies by a . Thus $(\lg 2k) - 1 = (\lg(2k + 1)) - 1$ squarings and $s_2(2k) + 1 = s_2(2k + 1)$ multiplications by a are performed. Similarly, if e is even, say $e = 2k$, then the algorithm first computes a^k and squares the result. Thus by induction $\lg k = (\lg 2k) - 1$ squarings and $s_2(k) = s_2(2k)$ multiplications are performed.

Finally, by observing that $O(\lg e)$ squarings or multiplications of numbers of size $O(\lg n)$ are performed, the time bound of $O((\lg e)(\lg n)^2)$ bit operations follows. ■

It is easy to give two iterative versions of POWER; one based on reading the bits of the exponent from right to left, and the other from left to right. See Exercise 3.

Note that the power algorithm works in any semigroup, not just the abelian multiplicative semigroup of integers modulo n . For example, by replacing multiplication modulo n with integer addition, we get a polynomial-time algorithm for multiplication.

The power algorithm can also be useful in many instances where its applicability is not immediately obvious. For example, let us consider the problem of computing the k -th Fibonacci number (mod n), where k is large. If we use the fact that $F_k = F_{k-1} + F_{k-2}$, then an iterative approach uses $O(k)$ additions of numbers smaller than n , which is not a very efficient method. We can rephrase the problem in terms of matrices if we observe that

$$\begin{bmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{bmatrix} = \begin{bmatrix} F_k & F_{k-1} \\ F_{k-1} & F_{k-2} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

Hence by induction, we have

$$\begin{bmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^k.$$

Now if we replace multiplication of integers in the power algorithm with multiplication of 2×2 matrices, we see that we can compute $F_k \bmod n$ using $O((\lg k)(\lg n)^2)$ bit operations.

Other applications of the power algorithm can be found in Exercises 5–9.

We note that the power algorithm is most useful when the representation of a^e is not significantly larger than that of a . This is the case in the examples above, where all intermediate results are reduced modulo n before continuing.

5.5 The Chinese Remainder Theorem

In this section we discuss the computational complexity of the Chinese remainder theorem, the fundamental tool for converting between a single congruence and a system of congruences with smaller moduli.

DEFINITION 5.5.1 We say that $m_1, m_2, \dots, m_k$ are *relatively prime in pairs* iff $\gcd(m_i, m_j) = 1$ for $i \neq j$.

THEOREM 5.5.2 (CHINESE REMAINDER THEOREM, VERSION 1) Let $m_1, m_2, \dots, m_k$ be positive integers, relatively prime in pairs. Let $m = m_1 m_2 \cdots m_k$. Then, given integers $x_1, x_2, \dots, x_k$, there is an integer x such that

$$x \equiv x_i \pmod{m_i},$$

and this integer x is uniquely determined, modulo m .

Proof Suppose we could find integers e_i , $1 \leq i \leq k$ such that

$$e_i \equiv \begin{cases} 1 \pmod{m_i}, \\ 0 \pmod{m_j}, \end{cases} \quad \text{if } j \neq i. \quad (5.2)$$

Then if

$$x \equiv \sum_{1 \leq i \leq k} x_i e_i,$$

we would have $x \equiv x_i \pmod{m_i}$ for $1 \leq i \leq k$.

How do we find such integers e_i ? Suppose we define

$$f_i = \frac{m}{m_i}.$$

Then $f_i \equiv 0 \pmod{m_j}$ for $j \neq i$. Now let $g_i \equiv f_i^{-1} \pmod{m_i}$; the inverse exists because the m_j are relatively prime in pairs. If we let $e_i = f_i g_i$, then e_i satisfies equation (5.2).

Thus we have found an integer x satisfying the given congruences.

To see that such an integer is uniquely determined modulo m , let x and y both satisfy the congruences. Then $x - y \equiv 0 \pmod{m_i}$ for $1 \leq i \leq k$. Since the m_i are pairwise relatively prime, $x \equiv y \pmod{m}$. ■

COROLLARY 5.5.3 Given moduli $m_1, m_2, \dots, m_k > 1$, relatively prime in pairs, and integers $x_1, x_2, \dots, x_k$, we can find a solution to the congruences $x \equiv x_i \pmod{m_i}$, $1 \leq i \leq k$, using $O((\lg m)^2)$ bit operations, where $m = m_1 m_2 \cdots m_k$.

Proof We note that the restriction to moduli greater than 1 is to avoid the case where, for example, all the moduli are 1 and there are many of them. In this case, $m = m_1 m_2 \cdots m_k = 1$, and we could not solve the congruences (or even examine all of them) within the stated time bound.

To achieve the desired complexity bound, note that the solution given above consists of (i) computing m and $f_i = m/m_i$; (ii) computing $g_i = f_i^{-1} \pmod{m_i}$; (iii) computing $e_i = f_i g_i$ and (iv) computing $x = \sum_i x_i e_i$.

By Exercise 3.3, we can compute m using $O((\lg m)^2)$ bit operations. It costs $O((\lg m)(\lg m_i))$ to compute f_i ; summing this from $i = 1$ to k gives us $O((\lg m)(k + \log m)) = O((\lg m)^2)$, since $k = O(\log m)$. Similar results hold for (ii)–(iv). ■

The previous version of the Chinese remainder theorem can be regarded as a statement about solutions to certain equations. The next version is a *structure* theorem.

THEOREM 5.5.4 (CHINESE REMAINDER THEOREM, VERSION 2) Let $m_1, m_2, \dots, m_k$ be positive integers, relatively prime in pairs. Then

$$\mathbb{Z}/(m_1 m_2 \cdots m_k) \cong \mathbb{Z}/(m_1) \oplus \mathbb{Z}/(m_2) \oplus \cdots \oplus \mathbb{Z}/(m_k).$$

Proof Let $f_i : \mathbb{Z} \rightarrow \mathbb{Z}/(m_i)$ be the map that takes n to its equivalence class $n \pmod{m_i}$. We construct a map from $\mathbb{Z}$ to $\mathbb{Z}/(m_1) \oplus \mathbb{Z}/(m_2) \oplus \cdots \oplus \mathbb{Z}/(m_k)$ as follows: $f(n) = (f_1(n), f_2(n), \dots, f_k(n))$. It is easily checked that f is a ring homomorphism.

What is the image of f ? We have $(a_1, a_2, \dots, a_k) = f(n)$ iff $f_i(n) = a_i$ for $1 \leq i \leq k$. By the first version of the Chinese remainder theorem, we know that such an n always exists. Thus f is onto.

What is the kernel of f ? We have $f(n) = 0$ iff $n \equiv 0 \pmod{m_i}$ for $1 \leq i \leq k$, iff $n \equiv 0 \pmod{m_1 m_2 \cdots m_k}$. Thus the kernel of f is the ideal $(m_1 m_2 \cdots m_k)$.

Hence the two rings are isomorphic. ■

This version of the Chinese remainder theorem can be interpreted in terms of “data structures”. We can represent an integer i in the range $0 \leq i < m = m_1 m_2 \cdots m_k$ in two ways: (i) as an integer modulo m , or (ii) as a list of integers, modulo each m_j . It is easy to see that we can convert from representation (i) to representation (ii) using $O((\lg m)^2)$ bit

operations, and by Corollary 5.5.3, we can convert from representation (ii) to representation (i) in the same time bound.

The Chinese remainder theorem can be generalized to cover the case where the moduli are not necessarily relatively prime, as follows:

THEOREM 5.5.5 The system of congruences

$$x \equiv x_i \pmod{m_i},$$

$1 \leq i \leq k$ has a solution iff $x_i \equiv x_j \pmod{\gcd(m_i, m_j)}$ for all $i \neq j$. If the solution exists, it is unique $\pmod{\text{lcm}(m_1, m_2, \dots, m_k)}$.

Proof Suppose $x \equiv x_i \pmod{m_i}$ and $x \equiv x_j \pmod{m_j}$. Then clearly $x \equiv x_i \pmod{\gcd(m_i, m_j)}$ and $x \equiv x_j \pmod{\gcd(m_i, m_j)}$, and we conclude $x_i \equiv x_j \pmod{\gcd(m_i, m_j)}$.

To prove uniqueness of the solution, note that if $x \equiv x_i \pmod{m_i}$ for $1 \leq i \leq k$ and $y \equiv x_i \pmod{m_i}$ for $1 \leq i \leq k$, then $x - y \equiv 0 \pmod{m_i}$; hence $x - y$ is a multiple of $\text{lcm}(m_1, m_2, \dots, m_k)$.

To complete the proof, we must show that a solution exists: Gauss gave a constructive method, which works by successively reducing a pair of congruences to a single congruence. Suppose we have

$$x \equiv x_1 \pmod{m_1}$$

$$x \equiv x_2 \pmod{m_2}.$$

Then $x = x_1 + tm_1$ for some t . Then $x_1 + tm_1 \equiv x_2 \pmod{m_2}$; hence $tm_1 \equiv x_2 - x_1 \pmod{m_2}$. Now let $d = \gcd(m_1, m_2)$; then $d \mid x_2 - x_1$. Thus

$$t \frac{m_1}{d} \equiv \frac{x_2 - x_1}{d} \pmod{m_2/d}.$$

By Theorem 5.3.1, this congruence has a unique solution $t \equiv t_1 \pmod{m_2/d}$. Substituting $t = t_1 + jm_2/d$ in $x = x_1 + tm_1$, we find $x = x_1 + t_1m_1 + jm_1m_2/d$. Hence $x \equiv x_1 + t_1m_1 \pmod{\text{lcm}(m_1, m_2)}$. By repeating this construction $k - 1$ times, we find the solution to a system of k simultaneous congruences. ■

COROLLARY 5.5.6 Let $m_1, m_2, \dots, m_k$ be integers, each ≥ 2 , and define $m = m_1m_2 \cdots m_k$, and $m' = \text{lcm}(m_1, m_2, \dots, m_k)$. Given the system S of congruences $x \equiv x_i \pmod{m_i}$, $1 \leq i \leq k$, we can determine if S has a solution, using $O((\lg m)^2)$ bit operations, and if so, we can find the unique solution $\pmod{m'}$, using $O((\lg m)^2)$ bit operations.

Proof Left to the reader as Exercise 4. ■

Although implementing the Chinese remainder theorem can be done in polynomial time, it turns out that a similar problem, which we might call the “anti-Chinese remainder theorem”, is $\mathcal{NP}$ -complete:

THEOREM 5.5.7 The following decision problem is $\mathcal{NP}$ -complete: given k and a list of k pairs

$$(x_1, m_1), (x_2, m_2), \dots, (x_k, m_k),$$

determine if there exists x such that $x \not\equiv x_i \pmod{m_i}$ for $1 \leq i \leq k$.

Proof Note that we do *not* require that the m_i be relatively prime in pairs.

First, note that the problem is in $\mathcal{NP}$, as a nondeterministic algorithm can guess such an x and verify it satisfies the k incongruences in polynomial time. If such an x exists, it satisfies $0 \leq x < \text{lcm}(m_1, m_2, \dots, m_k)$, and hence $\lg x$ is bounded by a polynomial in the length of the input.

To show the problem is $\mathcal{NP}$ -complete, we use reduction from 3-SAT. Let the variables be $y_1, y_2, \dots, y_t$, and let F be a formula which is the conjunction of n clauses of the form $C_i = (z_{a_i} \vee z_{b_i} \vee z_{c_i})$, where z_{a_i} is either y_{a_i} or $\overline{y_{a_i}}$, and the same for b_i and c_i . We encode a truth assignment to the variables as both a bit vector $(e_1, e_2, \dots, e_t)$ and an integer x via the correspondence $x \equiv e_i \pmod{p_i}$, where p_i is the i -th prime number. (From Theorem 8.8.4, we know that $p_t < t(\log t + \log \log t)$, for $t \geq 6$, so we can determine the first t primes via brute force in time polynomial in t .)

Now define a'_i to be 0 if $z_{a_i} = y_{a_i}$, and 1 if $z_{a_i} = \overline{y_{a_i}}$, and similarly for the sequences b'_i and c'_i . We define a certain integer x_i for each clause C_i , $1 \leq i \leq n$, as follows:

$$x_i \equiv a'_i \pmod{p_{a_i}}$$

$$x_i \equiv b'_i \pmod{p_{b_i}}$$

$$x_i \equiv c'_i \pmod{p_{c_i}}$$

We can find such an integer x_i in the range $[0, p_{a_i}p_{b_i}p_{c_i})$ in polynomial time by the Chinese remainder theorem.

Finally, our system S of linear incongruences is given by

$$x \not\equiv 2 \pmod{3}$$

$$x \not\equiv 2, 3, 4 \pmod{5}$$

$$\vdots$$

$$x \not\equiv 2, 3, \dots, p_t - 1 \pmod{p_t}$$

$$\begin{aligned}
x &\not\equiv x_1 \pmod{p_{a_1}p_{b_1}p_{c_1}} \\
x &\not\equiv x_2 \pmod{p_{a_2}p_{b_2}p_{c_2}} \\
&\vdots \\
x &\not\equiv x_n \pmod{p_{a_n}p_{b_n}p_{c_n}}
\end{aligned}$$

Note that there are $n + \sum_{2 \leq p \leq p_t} p - 2 = O(n + t^3)$ incongruences.

Now we prove that the formula F is satisfiable if and only if this system of incongruences has a solution. The first few incongruences are needed to ensure that $x \equiv 0, 1 \pmod{p_i}$. Now let an assignment be $(y_1, y_2, \dots, y_n) = (y'_1, y'_2, \dots, y'_n)$. Then clause C_i is satisfied if and only if $(y'_{a_i}, y'_{b_i}, y'_{c_i}) \neq (a'_i, b'_i, c'_i)$, which by our construction means $x \not\equiv x_i \pmod{p_{a_i}p_{b_i}p_{c_i}}$. ■

5.6 The Multiplicative Structure of $(\mathbb{Z}/(n))^*$

In this section we examine the multiplicative group $(\mathbb{Z}/(n))^*$. We start with a simple lemma:

LEMMA 5.6.1 Suppose $R_1, R_2, \dots, R_k$ are rings and $S = R_1 \oplus R_2 \oplus \dots \oplus R_k$. Then $S^* = R_1^* \times R_2^* \times \dots \times R_k^*$.

Proof Suppose $u \in S$ is a unit. Then there is a $v \in S$ such that $uv = (1, 1, \dots, 1)$, the identity element for S . Hence if $u = (u_1, u_2, \dots, u_k)$ and $v = (v_1, v_2, \dots, v_k)$, we have $u_i v_i = 1$ for $1 \leq i \leq k$. Thus u_i is a unit for each i .

Conversely, if u_i is a unit for each i , then there exists v_i such that $u_i v_i = 1$ for each i . Then $u = (u_1, u_2, \dots, u_k)$ is a unit, since if $v = (v_1, v_2, \dots, v_k)$, then $uv = (1, 1, \dots, 1)$. ■

We now apply this lemma to the multiplicative structure of $(\mathbb{Z}/(n))^*$. Let the prime factorization of n be

$$n = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}.$$

Then from Theorem 5.5.4 we have

$$\mathbb{Z}/(n) \cong \mathbb{Z}/(p_1^{e_1}) \oplus \mathbb{Z}/(p_2^{e_2}) \oplus \dots \oplus \mathbb{Z}/(p_k^{e_k}).$$

Then from the preceding lemma we have

$$(\mathbb{Z}/(n))^* \cong (\mathbb{Z}/(p_1^{e_1}))^* \times (\mathbb{Z}/(p_2^{e_2}))^* \times \dots \times (\mathbb{Z}/(p_k^{e_k}))^*.$$

Thus to determine the multiplicative structure of $(\mathbb{Z}/(n))^*$, it suffices to determine the structure of $(\mathbb{Z}/(p^e))^*$ for each prime power $p^e \parallel n$.

We say a group G is *cyclic* if it is generated by a single element. In the case where $G = (\mathbb{Z}/(n))^*$, such a generating element is frequently called a *primitive root*.

THEOREM 5.6.2 Let p, e be integers with p prime and $e \geq 1$. Then $(\mathbb{Z}/(p^e))^*$ is an abelian group with $p^{e-1}(p-1)$ elements. If $p \geq 3$, then $(\mathbb{Z}/(p^e))^*$ is a cyclic group. If $p = 2$, then $(\mathbb{Z}/(p^e))^*$ is cyclic for $e \leq 2$. For $e > 2$, $(\mathbb{Z}/(p^e))^*$ is the direct product of two cyclic groups, one of order 2 and the other of order 2^{e-2} ; it is generated by the elements -1 and 5.

Proof See Exercises 14–21.

THEOREM 5.6.3 Let n be an integer > 1 . Then $(\mathbb{Z}/(n))^*$ has a primitive root iff $n = 2, 4, p^e, 2p^e$, where $e \geq 1$ and p is an odd prime.

Proof We know from Theorem 5.6.2 that $(\mathbb{Z}/(2^e))^*$ is cyclic if $e = 1$ or $e = 2$. We also know from Theorem 5.6.2 that $(\mathbb{Z}/(p^e))^*$ is cyclic if p is an odd prime. Also, since $(\mathbb{Z}/(2p^e))^* = (\mathbb{Z}/(2))^* \times (\mathbb{Z}/(p^e))^*$, we see that $(\mathbb{Z}/(2p^e))^*$ is cyclic.

Now assume n is not of the required form. Then (i) $n = 2^e$, for $e > 2$; or (ii) we can write $n = rs$, where $\gcd(r, s) = 1$ and $r, s > 2$. In the first case, we know from Theorem 5.6.2 that $(\mathbb{Z}/(2^e))^*$ is not cyclic, as it has at least two elements of order 2. In the second case, we know that $(\mathbb{Z}/(n))^* \cong (\mathbb{Z}/(r))^* \times (\mathbb{Z}/(s))^*$; and $\varphi(r)$ and $\varphi(s)$ are both even. Hence again $(\mathbb{Z}/(n))^*$ has at least two elements of order 2, and thus is not cyclic. ■

5.7 Quadratic Residues

DEFINITION 5.7.1 Suppose m and n are positive integers. Suppose a is an integer and $\gcd(a, n) = 1$. Then we say a is an m -th *power residue* (mod n) iff $x^m \equiv a \pmod{n}$ is solvable.

THEOREM 5.7.2 Suppose $(\mathbb{Z}/(n))^*$ is cyclic and $\gcd(a, n) = 1$. Then a is an m -th power residue (mod n) iff $a^{\varphi(n)/d} \equiv 1 \pmod{n}$, where $d = \gcd(m, \varphi(n))$.

Proof Let g be a generator for $(\mathbb{Z}/(n))^*$, and let $a \equiv x^m \pmod{n}$. Then we can write $a \equiv g^b$ and $x \equiv g^y$. Then $g^{my} \equiv g^b \pmod{n}$, which implies that $my \equiv b \pmod{\varphi(n)}$. By Theorem 5.3.1, this implies that $d \mid b$. Hence $a^{\varphi(n)/d} \equiv g^{b\varphi(n)/d} \equiv 1 \pmod{n}$.

Conversely, suppose $a^{\varphi(n)/d} \equiv 1 \pmod{n}$. Then $g^{b\varphi(n)/d} \equiv 1 \pmod{n}$; hence $\varphi(n) \mid b\varphi(n)/d$ and so $d \mid b$. By Theorem 5.3.1, we can find integers e, f such that $em + f\varphi(n) = b$, and so $(g^e)^m \equiv g^b \equiv a \pmod{n}$. ■

A second power residue (mod n) is called a *quadratic residue*. If $\gcd(a, n) = 1$ and a is not a quadratic residue, then it is called a *quadratic nonresidue* (mod n). For example, the quadratic residues of 7 are 1, 2, and 4, and the quadratic nonresidues are 3, 5, and 6.

5.11 Notes on Chapter 5

5.2

For a discussion of hardware and software techniques intended to speed up multiplication (mod n), see Head [1980]; Norris and Simmons [1981]; Blakley [1983]; Brickell [1983]; P. Montgomery [1985]; Kurihara [1985]; P. Baker [1987]; Gibson [1988]; Kawamura and Hirano [1988]; Kurihara [1989]; Walter and Eldridge [1990]; Morita [1990a, 1990b]; Shand, Bertin, and Vuillemin [1990];

Alia and Martinelli [1991]; Even [1991]; Eldridge [1991]; Walter [1991, 1992, 1995]; N. Takagi [1991, 1992, 1993]; N. Takagi and Yajima [1992]; Skavantzios and Rao [1992]; Rao and Skavantzios [1992]; Iwamura, Matsumoto, and Imai [1993a]; Kornerup [1993].

5.3

From the results of Knuth [1971] and Schönhage [1971], it follows that we can compute $a^{-1} \bmod n$ for $1 \leq a < n$ in $O((\lg n)(\lg \lg n)^2(\lg \lg \lg n))$ steps. This method is likely to be unsuitable for practical use. However, a dissenting view can be found in Schönhage, Grotfeld, and Vetter [1994].

A major unsolved problem is to determine the parallel complexity of computing inverses in $(\mathbb{Z}/(n))^*$. It is not known to be in $\mathcal{NC}$, nor is it known to be $\mathcal{P}$ -complete.

5.4

The power algorithm or binary method for exponentiation goes back to at least 200 B.C.; see Datta and Singh [1935, p. 76]. A similar method can be used to multiply two numbers and is sometimes known as “Russian peasant multiplication.” The binary method is mentioned in the following 18th century works: Euler [1761]; Lagrange [1769]; Legendre [1798]. Pocklington [1910] may have been the first to remark explicitly that the binary method for computing $a^e \bmod n$ uses $O(\log e)$ multiplications $(\bmod n)$.

Depending on the cost of a multiplication, the binary method may not necessarily be the cheapest. Other algorithms may be faster for the symbolic computation of the power of a polynomial. See, for example, Gentleman [1972], Fateman [1974a, 1974b], and McCarthy [1977, 1986].

Willoner and Chen [1981] discussed computing $a^e \bmod n$ efficiently for many values of a , where n is fixed. Brickell, Gordon, McCurley and Wilson [1993] showed how to calculate g^n for $n < N$ efficiently.

The constant factor in the time bound for exponentiation $\bmod n$ can be reduced by “ k -ary” methods and “addition chains”. See, for example, Scholz [1937]; Brauer [1939]; Utz [1953]; Erdős [1960]; Gioia, Subbarao, and Sugunama [1962]; Straus [1964]; Cottrell [1973]; Thurber [1973a, 1973b, 1976, 1993]; Schönhage [1975]; Vegh [1975]; Lipton and Dobkin [1976]; Pippenger [1976]; Yao [1976]; van Leeuwen [1977]; Gioia and Subbarao [1978]; Dobkin and Lipton [1980]; Pippenger [1980]; Downey, Leong, and Sethi [1981]; Olivos [1981]; Knuth [1981, Sect. 4.6.3]; Semba [1983]; Volger [1985]; Tsai and Chin [1987]; Bergeron, Berstel, Brlek and Duboc [1989]; Subbarao [1989]; Bos and Coster [1990]; Morain and Olivos [1990]; Egecioglu and Koç [1990]; Yacobi [1991a]; Brlek and Mallette [1992]; Brlek, Castéran, Habsieger, and Mallette [1995]; Sauerbrey and Dietel [1993]; H. W. Lenstra [1993]; Aiello and Subbarao [1993]; and Bergeron, Berstel and Brlek [1994]. In de Melo and Svaiter [1995], the authors examine the basic properties of addition-subtraction-multiplication chains.

The following question is open: given integers a, b, i , can one compute the i -th bit of a^b in polynomial time? In von zur Gathen [1984a, 1987b] the question is attributed to A. Borodin.

There is no known $\mathcal{NC}$ algorithm for computing $a^e \pmod n$ in parallel. Von zur Gathen [1984a, 1987b] has given such an algorithm in the case where the modulus has only small prime factors. (See also Zeugmann [1992].)

For methods to compute $a^e \pmod n$ in hardware, see Findlay and Johnson [1990]; Orup and Kornerup [1991]; Sauerbrey [1993].

Kompella and Adleman [1991] have shown how to design an efficient “checker” for a modular exponentiation program.

A lower bound for modular exponentiation was proved by Meidanis [1991].

For efficient methods to compute Fibonacci numbers, and terms of linear recurrences, see Escott [1900]; J. Miller and Spencer Brown [1966]; Shortt [1978]; Wilson and Shortt [1980]; Urbanek [1980]; Gries and Levin [1980]; Pettorossi [1980]; Er [1983]; Protasi and Talamo [1989]; and Capocelli, Cerbone, Cull, and Holloway [1990]; Maeder [1991]. The most efficient method has been given by Fiduccia [1985].

The following problem is still open: given an order- k linear recurrence $S(i)$ and an integer n , can one determine in polynomial time whether there exists j such that $S(j) = n$? Partial results were obtained by Kannan and Lipton [1986]. The problem also is open when considered modulo p , as it is a generalization of the discrete logarithm. See Chapter 12.

5.5

A special case of the Chinese remainder theorem was given between 280 and 437 A.D. by Sun Tsü. See Needham [1959, pp. 33–34, 119–120]. See Dickson [1919, V, II, pp. 57–64] for early work on this theorem. Kangsheng [1988] has discussed the history of the Chinese remainder theorem. Also see Ku and Sun [1992]. For six proofs of the Chinese remainder theorem, from the ancients to the present day, see P. Davis and Hersh [1980, pp. 187–195].

Our constructive proof of the Chinese remainder theorem in Theorem 5.5.2 is from Gauss [1801, Art. 36]. The generalized Chinese remainder theorem, Theorem 5.5.5, is from Gauss [1801, Art. 32].

Theorem 5.5.7, the “anti-Chinese remainder theorem”, is from Stockmeyer and Meyer [1973].

For a systolic array to do Chinese remaindering, see Koç and Cappello [1989].

5.7

Euler’s criterion gives an $O((\lg p)^3)$ algorithm for determining the quadratic character of a , modulo p . Gauss, in writing about this method, said [1801, Art. 106]: , “... as soon as the numbers we are examining are even moderately large this criterion is practically useless because of the amount of calculation involved.” It is unclear why Gauss held this opinion. It seems unlikely that Gauss did not know the binary method of exponentiation, as the method appears in Euler [1761] (a paper that Gauss recommends) and was known by his contemporaries (see Lagrange [1769], Legendre [1798]). Note that the $O((\lg p)^2)$ algorithm of section 5.9 is superior to the algorithm based on Euler’s criterion, even for small numbers.

Burgess [1957, 1963a] proved that the least quadratic nonresidue of a prime p is bounded by $p^{1/(4\sqrt{e})+o(1)}$. We note that $a := 1/(4\sqrt{e}) \doteq .151633$. Burgess's result proves the existence of a *deterministic* algorithm to find a quadratic nonresidue using $O(p^{a+\epsilon})$ bit operations for every $\epsilon > 0$. This appears to be the best known unconditional result. For more on this problem, including algorithms that run quickly assuming the Extended Riemann hypothesis, see Chapter 8.

For an application of quadratic residues to hashing, see Maurer [1968] and Radke [1970].